

Advance Database Management

FINAL PROJECT ASSIGNMENT

Name: Fazeel Ahmed

Matricola: S5640427

Group: Single Member

Course: **ADVANCED DATA MANAGEMENT - 61884**

Professor: **Barbara Catania and Giovanna Guerrini**

Assignment Task 1

1. Identify a workload in Natural Language

Q1. Given a player identifier, determine the names of all stadiums in which the player has played at least once. **(Player)**

Q2. Given a player position and a specified market value, determine the names of all players satisfying those conditions. **(Player)**

Q3. Given a country name and a competition identifier, determine the types of all game events that occurred in the games of that competition. **(Competition)**

Q4. Given a player identifier and a specific year, determine all games played by that player during that year. **(Game)**

Q5. Given a specific year and a player name, determine the player's last season and the total number of minutes played during that year. **(Game)**

Verification of the Constraints:

Constraints	Result	Relevant queries
At least five structurally different operations	Satisfied	Q1–Q5
Three aggregation entities	Satisfied	(Player , Competition, and Game)
At least three queries selecting on two entities	Satisfied	Q2, Q3, Q4, Q5
At least two paths with length ≥ 2	Satisfied	Q1 and Q3
Partition-key case (c)	Satisfied	Q4 and Q5

2. Use the aggregate-oriented design methodology (STEP 1-2-3) to design a set of logical aggregates for the given domain and the selected workload.

Step 1 (Query Modeling):

Q1:

- E = Player
- LS = [Player(player_id)_!]
- LP = [Stadium(name)_PiPi]

Q1 (Player, [Player(player_id)_!], [Stadium(name)_PiPi])

Q2:

- E = Player
- LS = [Player(position)_!, PlayerValuation(market_value)_OF]
- LP = [Player(name)_!]

Q2 (Player, [Player(position)_!, PlayerValuation(market_value)_OF], [Player(name)_!])

Q3:

- E = Competition
- LS = [Competition(competition_id)_!, Country(name)_In]
- LP = [GameEvents(type)_OH]

Q3 (Competition, [Competition(competition_id)_! , Country(name)_In], [GameEvents(type)_OH])

Q4:

- E = Game
- LS = [Game(date)_!, Player(player_id)_Pi]
- LP = [Game(game_id, season)_!]

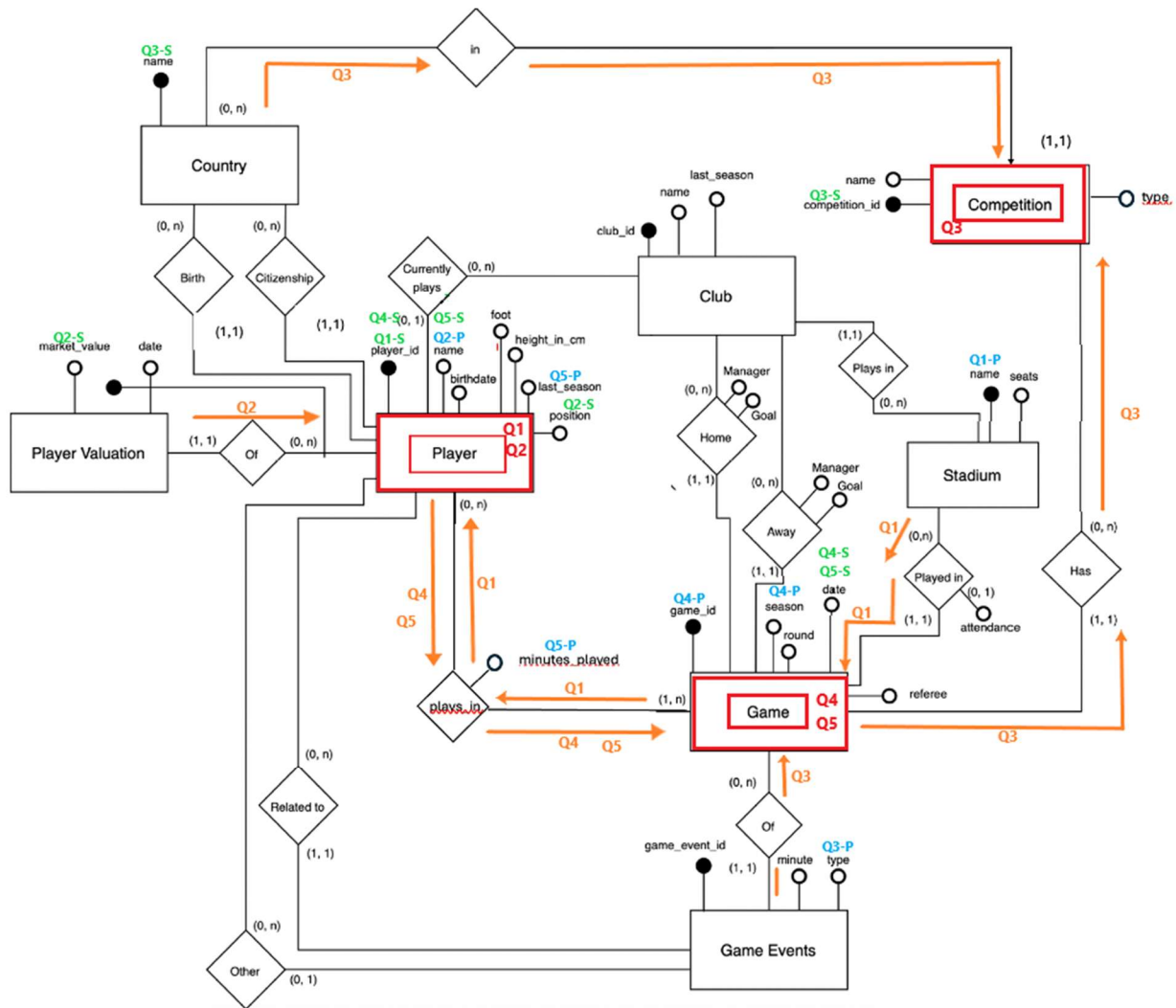
Q4 (Game, [Game(date)_! , Player(player_id)_Pi], [Game(game_id, season)_!])

Q5:

- E = Game
- LS = [Game(date)_!, Player(name)_Pi]
- LP = [Player(last_season)_Pi(minutes_played)]

Q5 (Game, [Game(date)_!, Player(name)_Pi], [Player(last_season)_Pi(minutes_played)])

Step 2 (Annotation of the ER Schema):



Step 3 (Translation into JSON Meta Notation):

Player Aggregate (Q1 and Q2):

Player =

```
{
  player_id, name, position,
  PlaterValuationOF: [ {marketvalue} ],
  plays_in:[ {Game: {game_id, name}}]
}
```

Competition Aggregate (Q3):

Competition =

```
{
  competition_id, country_name,
  Games: [ { Game: {game_id, game_event: [ {type} ] } } ]
}
```

Game Aggregate (Q4 and Q5):

```
Game =
{
  game_id, season, date,
  play_by:[ { player_id, name, last_season, minutes_played } ],
}
```

3. For each aggregate-oriented schema and each associated query, specify the query type (either batch or random access query).

- **Player Aggregate (Q1 and Q2):**

Random access query, because both queries contain selection entities (Player.player_id, Player.position, and PlayerValuation.market_value). Therefore, only a subset of the Player aggregates will be accessed, and no joining with other collections is required.

- **Competition Aggregate (Q3)**

Random access query (point query), because the query contains selection entities (Competition.competition_id and Country.name). Therefore, only a subset of the Competition aggregates will be accessed, and no joining with other collections is required.

- **Game Aggregate (Q4 and Q5)**

Random access query, because both queries contain selection entities (Game.date, Player.player_id, and Player.name). Therefore, only a subset of the Game aggregates will be accessed, and no joining with other collections is required.

All queries are Random Access Queries.

4. For each designed aggregate-oriented schema, specify a suitable partition key and a suitable super-identifier for the corresponding collection.

- **Player Aggregate (Q1 & Q2):**

Player = { player_id^P, name, position, PlayerValuationOF: [{market_value}], plays_in: [{Game: {game_id, name}}] }

Selection Attributes:

Q1 = {player_id}

Q2 = {position, market_value}

Case: (d) – The two queries do not share any common selection attribute.

Partition Key: player_id

Super-Identifier: player_id

Q1 is favoured because it directly searches a player using player_id. Since player_id is already the identifier and partition key, no extension is required.

- **Competition Aggregate (Q3)**

Competition = { competition_id^{p1}, country_name^{p2}, Games: [{ Game: {game_id, game_event: [{type}]} }] }

Selection Attribute: Q3 = {competition_id, country_name}

Case: (b) – All queries share the same selection attributes.

Partition Key: (competition_id, country_name)

Super-Identifier: (competition_id, country_name)

The original identifier competition_id is extended with country_name because the super-identifier must include the complete partition key.

- **Game Aggregate (Q4 & Q5)**

Game = { game_id, season, date^p, play_by: [{ player_id, name, last_season, minutes_played }] }

Selection Attributes:

Q4 = {date, player_id}

Q5 = {date, player_name}

Case: (c) – Both queries share the common selection attribute date, but their complete selection sets are different.

Partition Key: date

Super-Identifier: (date, game_id)

The original identifier game_id is extended with date because the super-identifier must include the partition key.

5. For each designed aggregate-oriented schema and each associated query, specify if it can be executed efficiently on the collection.

- **Player Aggregate (Q1 and Q2):**

Q1: Efficient.

The query contains an equality condition on the complete partition key player_id. Therefore, one node is accessed and the required aggregate is directly located.

Q2: Not efficient.

The query selects by position and market_value, but the partition key is player_id. Therefore, the system may need to access all nodes.

- **Competition Aggregate (Q3)**

Q3: Efficient.

The query contains equality conditions on both partition-key attributes, competition_id and country_name. Therefore, one node is accessed and the required aggregates are sequentially stored.

- **Game Aggregate (Q4 and Q5)**

Q4: Not fully efficient.

The condition on date identifies one node, but the query also contains a condition on player_id. Therefore, the relevant Game aggregates may not be sequentially stored on that node.

Q5: Not fully efficient.

The condition on date identifies one node, but the query also filters by player_name. Therefore, the relevant aggregates may not be sequentially stored on the node.

6. For each designed aggregate-oriented schema, identify the indexes to be created to make the execution of each query associated with the collection efficient (if any).

- **Player Aggregate (Q1 and Q2):**

A local ordered index must be created on the super-identifier: (player_id)

No additional index is required for Q1 because it selects by the partition key.

To improve Q2, a secondary local index should be created on: (position, market_value)

- **Competition Aggregate (Q3)**

No additional index is required because Q3 already uses all partition-key attributes and can be executed efficiently.

- **Game Aggregate (Q4 and Q5)**

A local ordered index must be created on the super-identifier, with the partition key first: (date, game_id)

To improve Q4, create a secondary local index on: (date, player_id)

To improve Q5, create a secondary local index on: (date, player_name)

DISCLOSURE: I used a Large Language Model (LLM) to improve the wording of the natural language queries and to help generate and understand the partition key and super-identifier, as applying the partition key selection cases to the queries was sometimes confusing. The final submission reflects my own understanding after reviewing the lecture material.